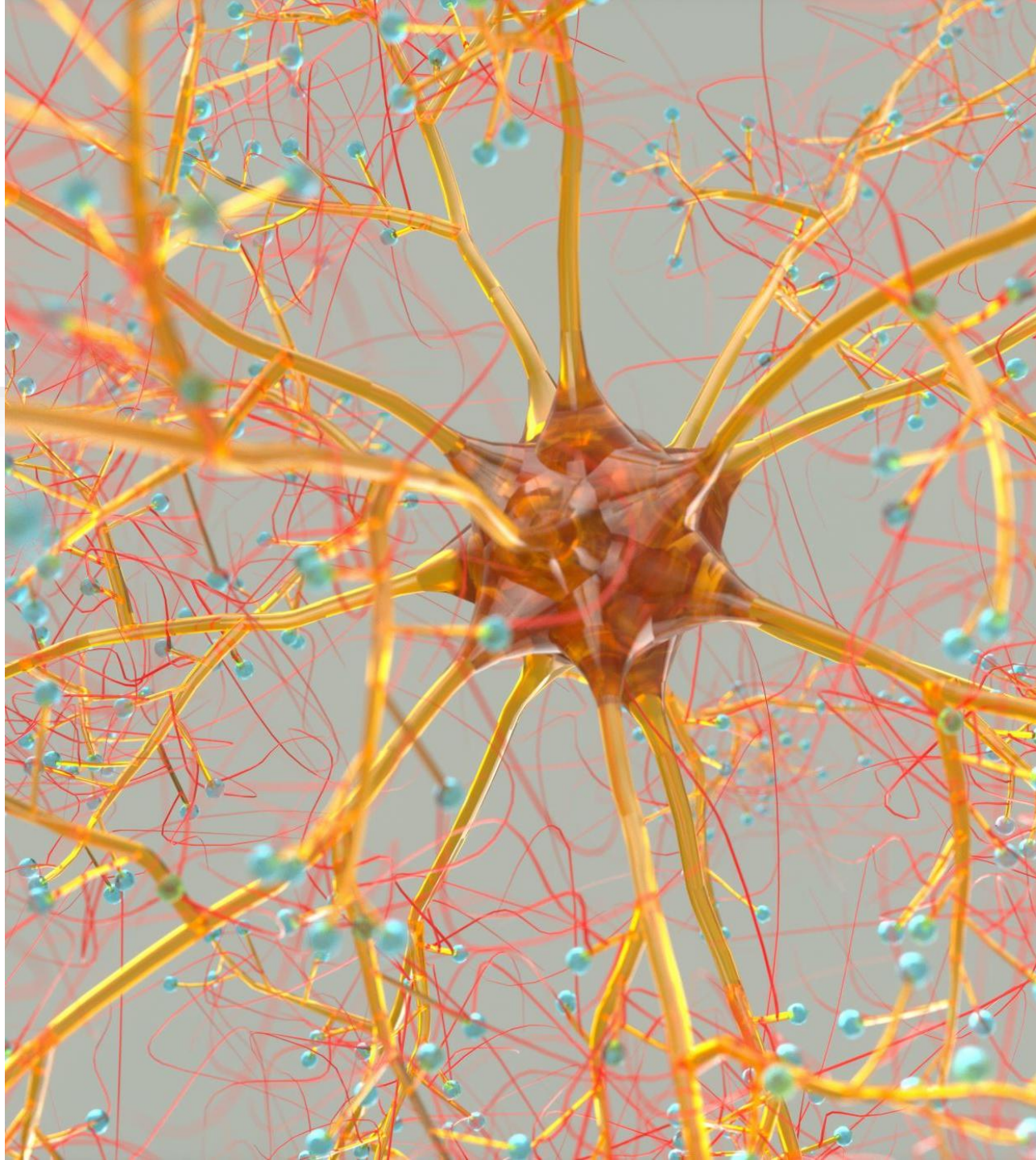
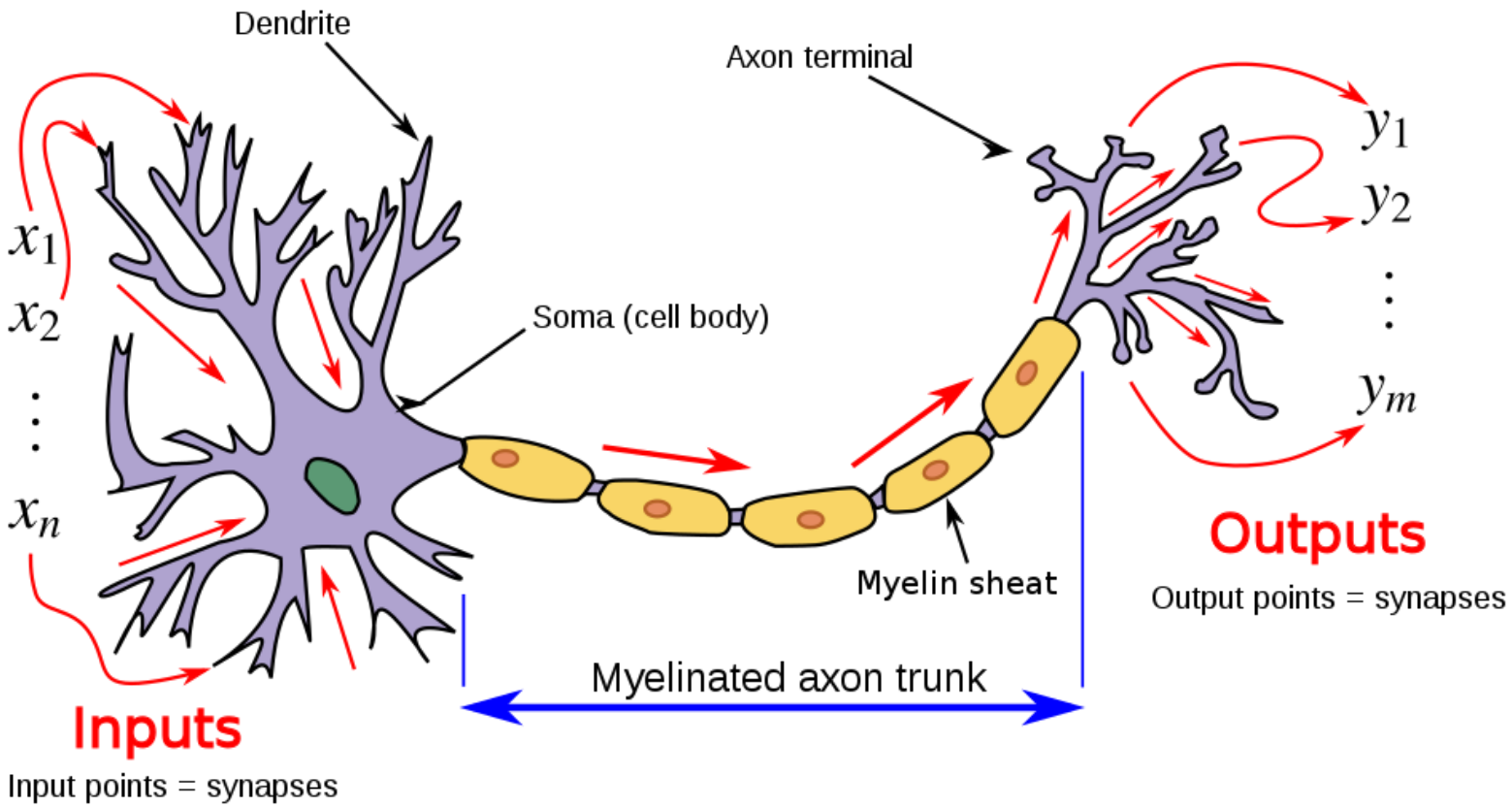
RETELE NEURONALE

Perceptonul

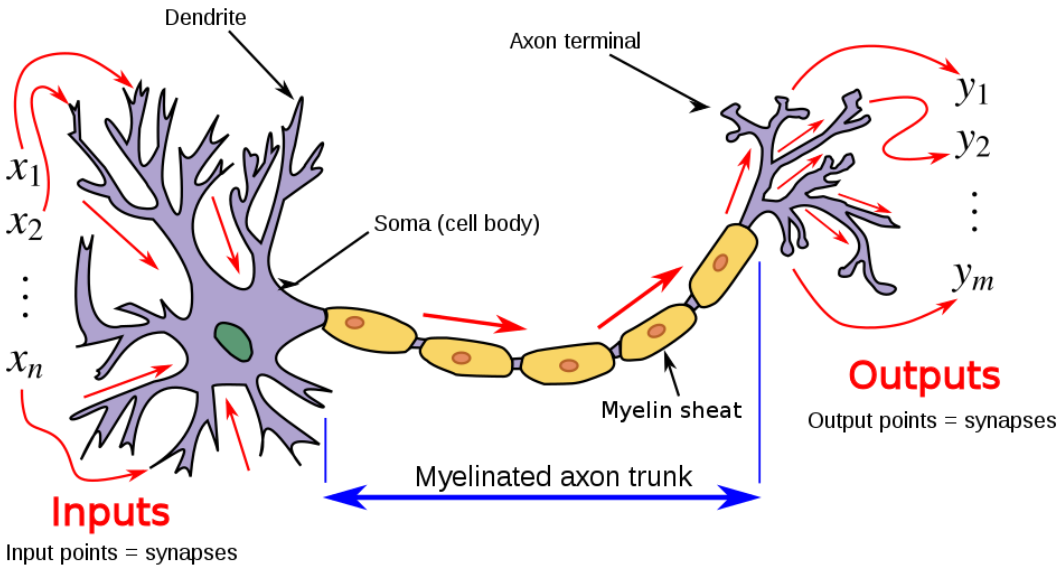
- Cel mai simplu model de retea neuronala – perceptonul
- Un singur neuron

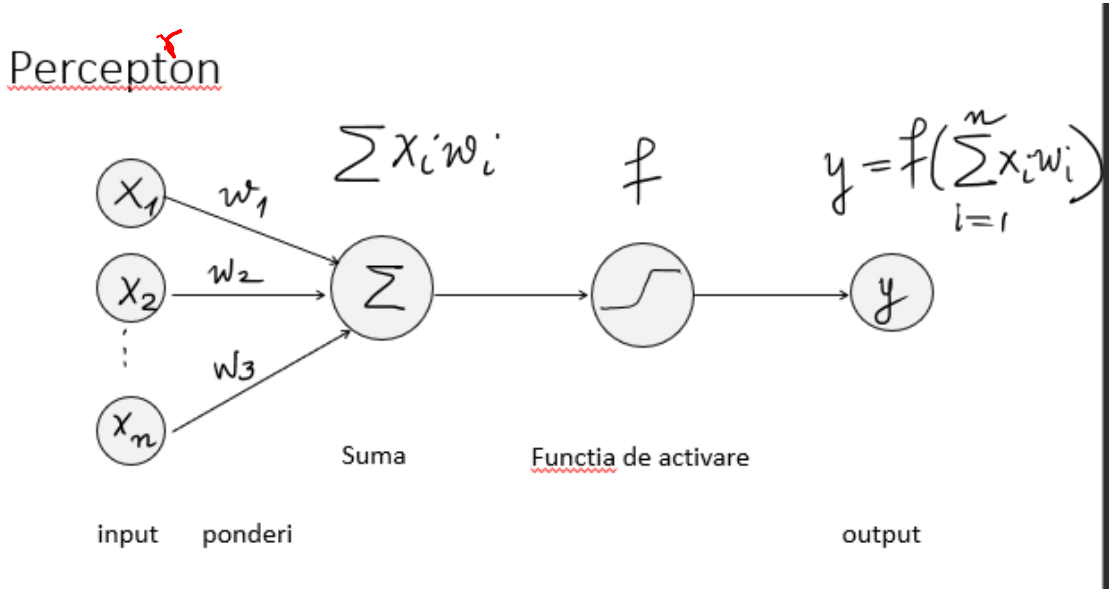


Schema unui neuron



Schema unui neuron vs. perceptron



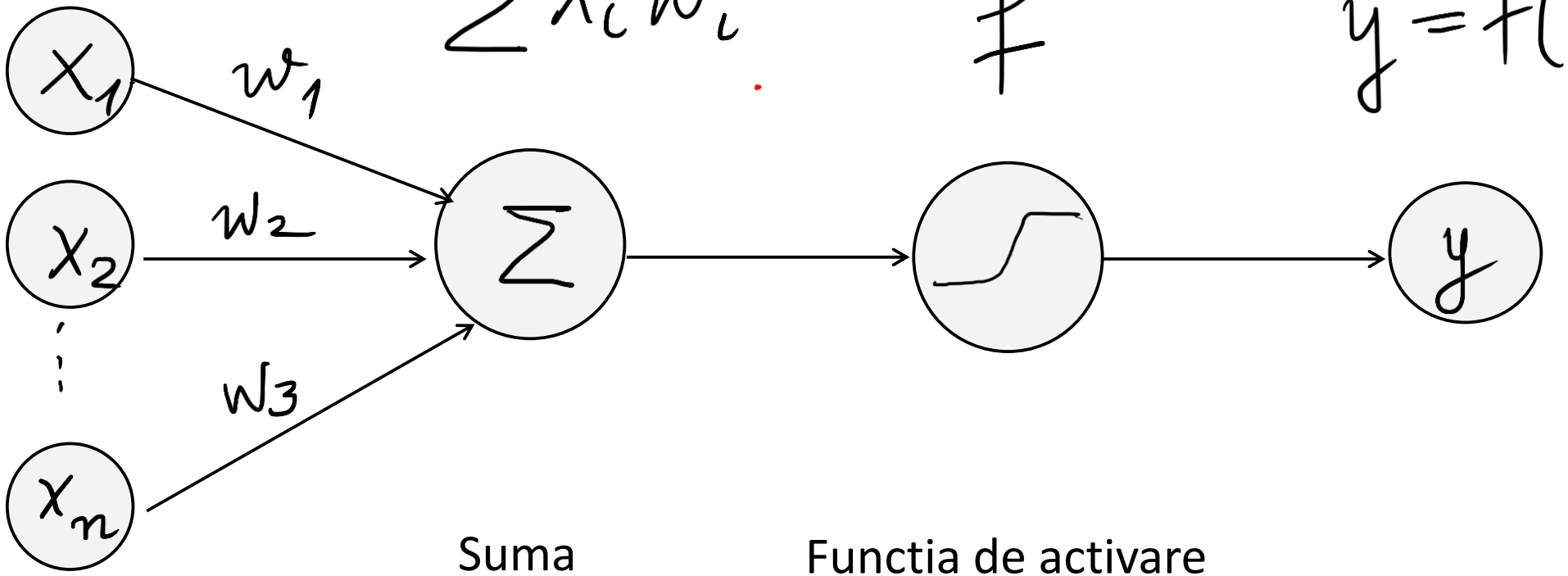


By Egm4313.s12 (Prof. Loc Vu-Quoc) - Own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=72801384

$z = x_1 w_1 + x_2 \cdot w_2 + \dots + x_n w_n$

Perceptron

$\sum x_i w_i$ $\quad f$ $\quad y = f\left(\sum_{i=1}^{n} x_i w_i\right)$



Suma

Functia de activare

$$f(x) = \begin{cases} 1, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

input ponderi output

Avem deci un set de intrare x1, x2, …xn. Fiecare x se inmulteste cu o pondere w. Se face apoi suma ponderata. Se aplica acestei sume functia de activare pe care o vom nota f si apoi se obtine valoarea de iesire y^ = f(suma(wi*xi)). Functia f este o functie neliniara pt ca in general, evenimentele nu sunt liniare, si deci prin intermediul acestei functii se introduce nonliniaritatea. Avem de ales dintr-o clasa de functii, nu sunt intamplator alese. Au anumite proprietati. Rezultatul obtinut il notam cu y.

Exemplu: decidem dacă un elev a învățat bine la un test

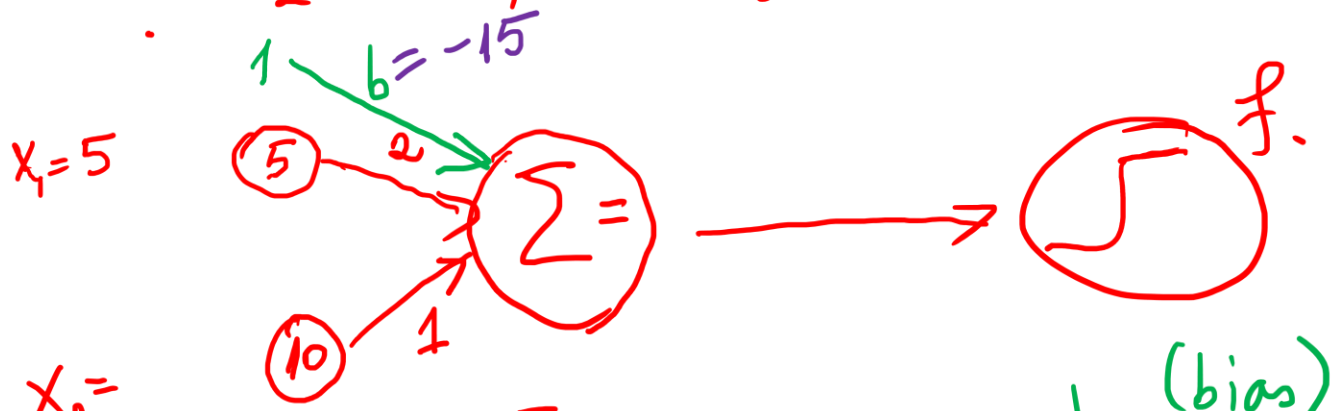
x_1 = nr. ore de învățare = 5 | $w_1 = 2$

x_2 = nr. probl. rezolvate = 10 | $w_2 = 1$

y val. de ieșire ≥ 15

A învățat Bine DA

nu



$\rightarrow$ y.

y = 1. DA a învățat bine.

$\Sigma = w_1 x_1 + w_2 x_2 + b$. (bias)

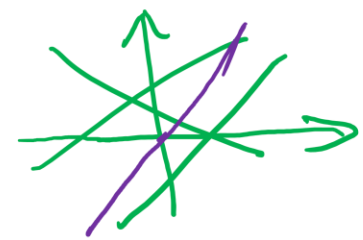
$= 2 \cdot 5 + 1 \cdot 10 = 20 - 15 = 5$

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

funcția treaptă

$$f(z) = \begin{cases} 1, & z \geq 15 \\ 0, & z < 15 \end{cases}$$

$y = mx + n$



Exemplu: decidem dacă un elev a învățat bine la un test

x_1 = nr. ore de învățare = 5 → $w_1 = 2$

x_2 = nr. probl. rezolvate = 10 → $w_2 = 1$

$b = -15$

y val. de ieșire ≥ 15

A învățat Bine DA

nu ... x_4

$\sum = w_1 x_1 + w_2 x_2 + b$ (bias)

$= 2 \cdot 5 + 1 \cdot 10 = 20 - 15 = 5$

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

funcția treaptă

$y = 1$. DA a învățat bine.

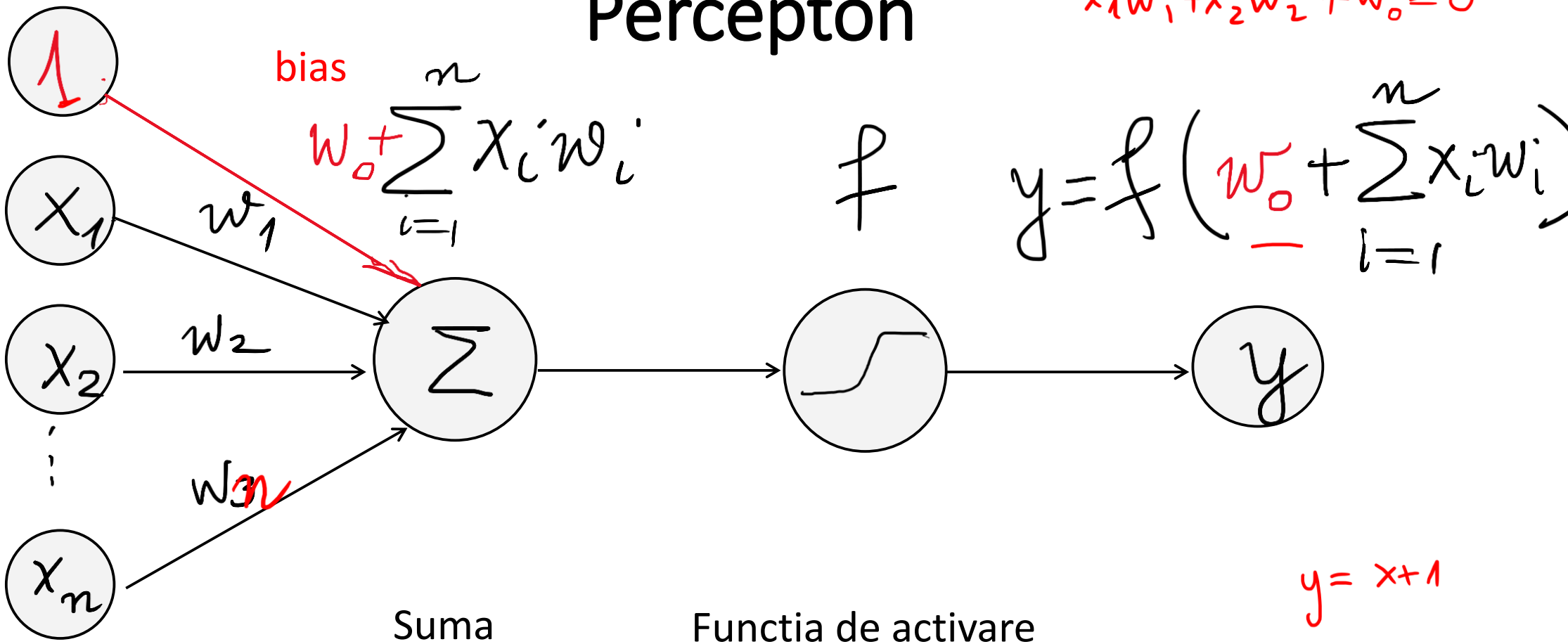
$$f(z) = \begin{cases} 1 \to z \geq 15 \\ 0, \quad z < 15 \end{cases}$$

$y = mx + n$

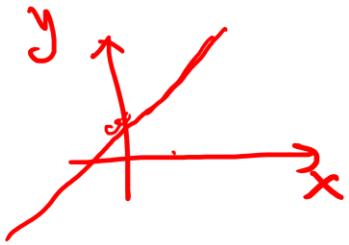
Percepton

$ax+by+c=0$

$x_1w_1+x_2w_2+w_0=0$

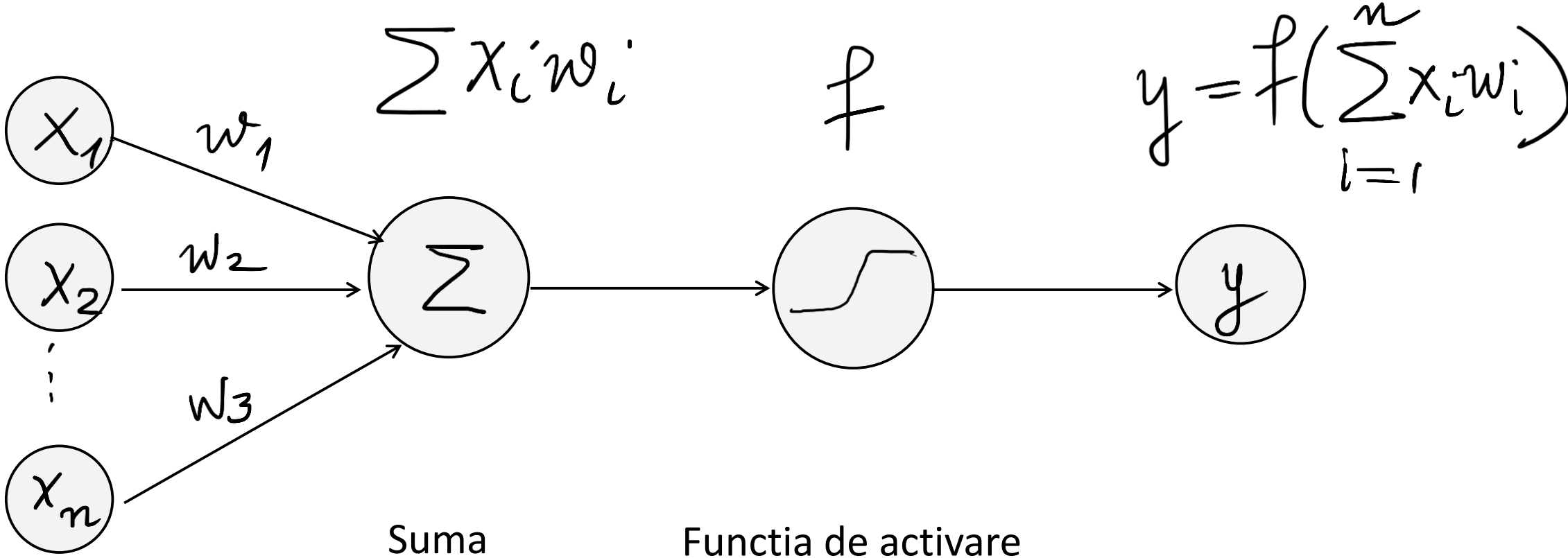


$y = x+1$



Suma ponderata nu este altceva decat o combinatie liniara a datelor de intrare. Echivalentul unei drepte in mai multe dimensiuni. Dar o dreapta are de fapt ecuatia Ax+by+c=0, deci are un termen liber, o constanta. Acest termen liber permite translatarea dreptei de-a lungul axei Oy. Pentru a surprinde si posibilitatea liniaritatii cu constanta de tip termen liber, la suma xi wi se adauga si o pondere w0 pe care am putea sa o vedem ca ponderea lui 1 (a constantei). Aceasta o numim bias. (abatere, reglare, shift). Si atunci la suma se adauga w0 si functia de activare se va aplica sumei w0+suma wixi.

Percepton original



Suma

Functia de activare
=
Functia treapta

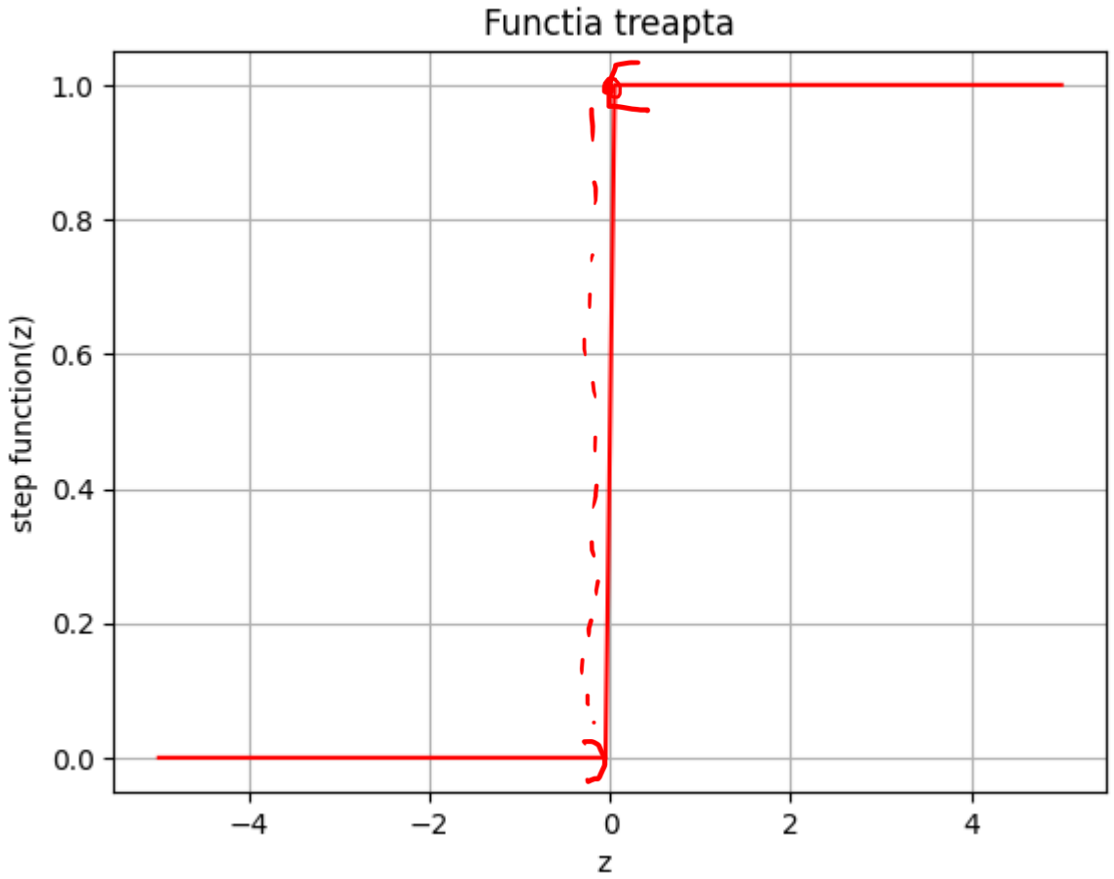
input ponderi output

Functii de activare

Functia treapta (step)

$$f(x) = \begin{cases} 1, daca\ x \geq 0 \\ 0, altfel \end{cases}$$

f: R-> {0,1}

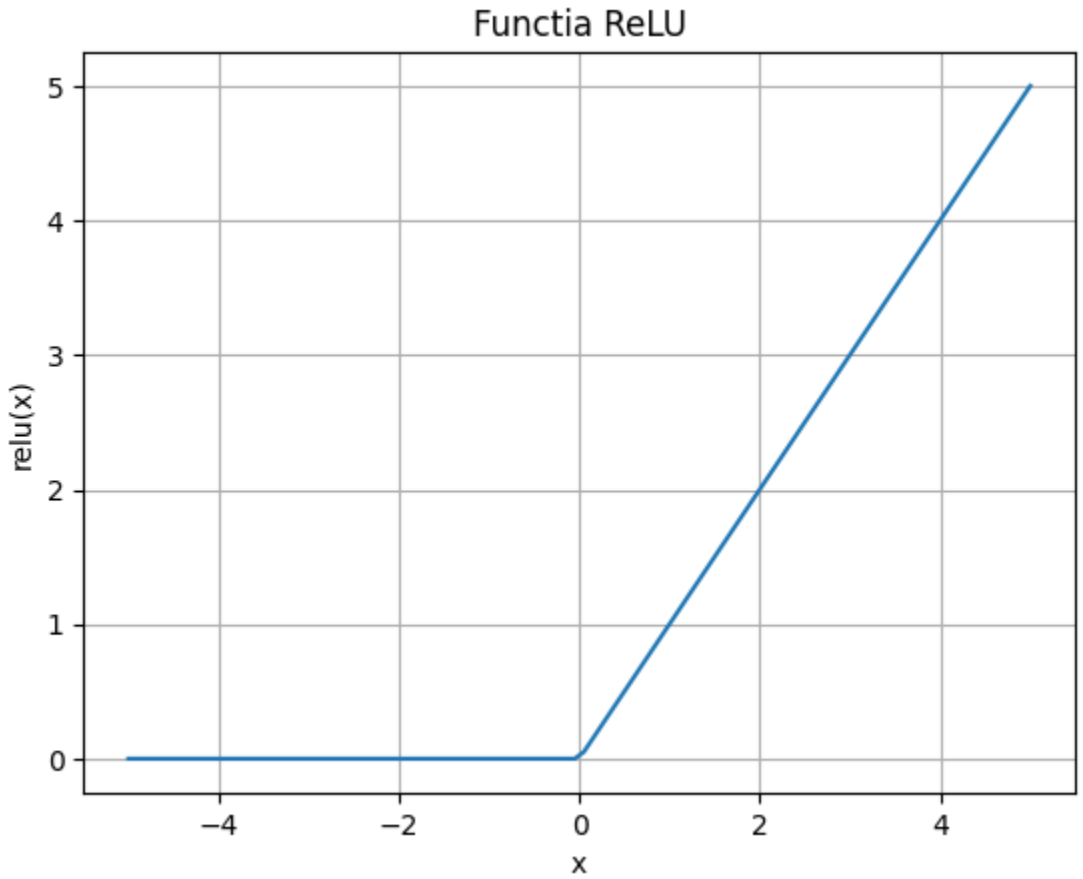


Functii de activare

Functia ReLU (Rectified Linear Unit)

$$f(x) = \begin{cases} x, daca\ x > 0 \\ 0, altfel \end{cases}$$

f: R-> [0,∞)



Functii de activare

Functia sigmoid (logistica)

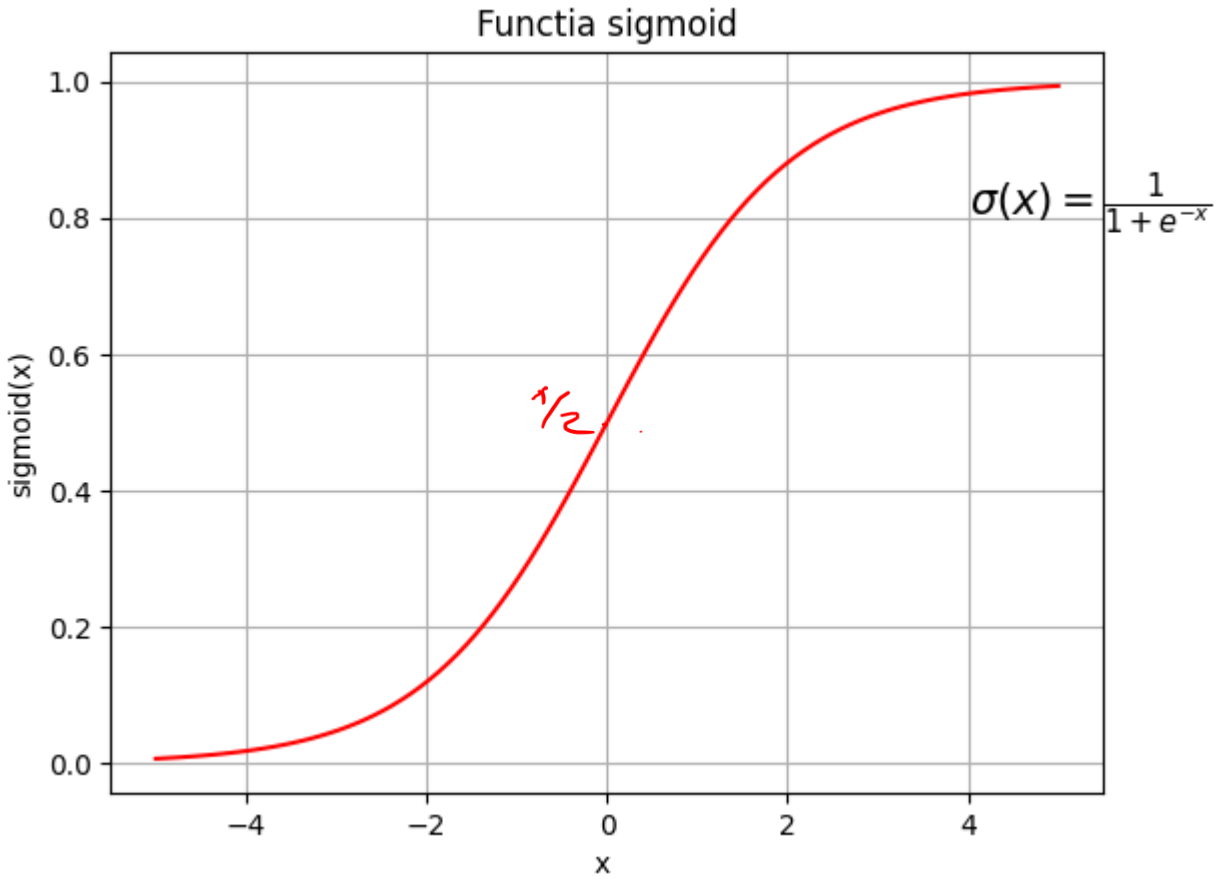
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

σ: R -> (0,1)

$\frac{1}{1+1} = \frac{1}{2}$

σ(0) = 0.5

Are proprietatea ca este definita pe R cu valori in (0,1). Se mai poate scrie e^x/(1+e^x). Sigma(0) = 0.5 care este punct de inflexiune pt graficul functiei.
Se foloseste cand ai nevoie sa calculezi probabilitati.



Functii de activare

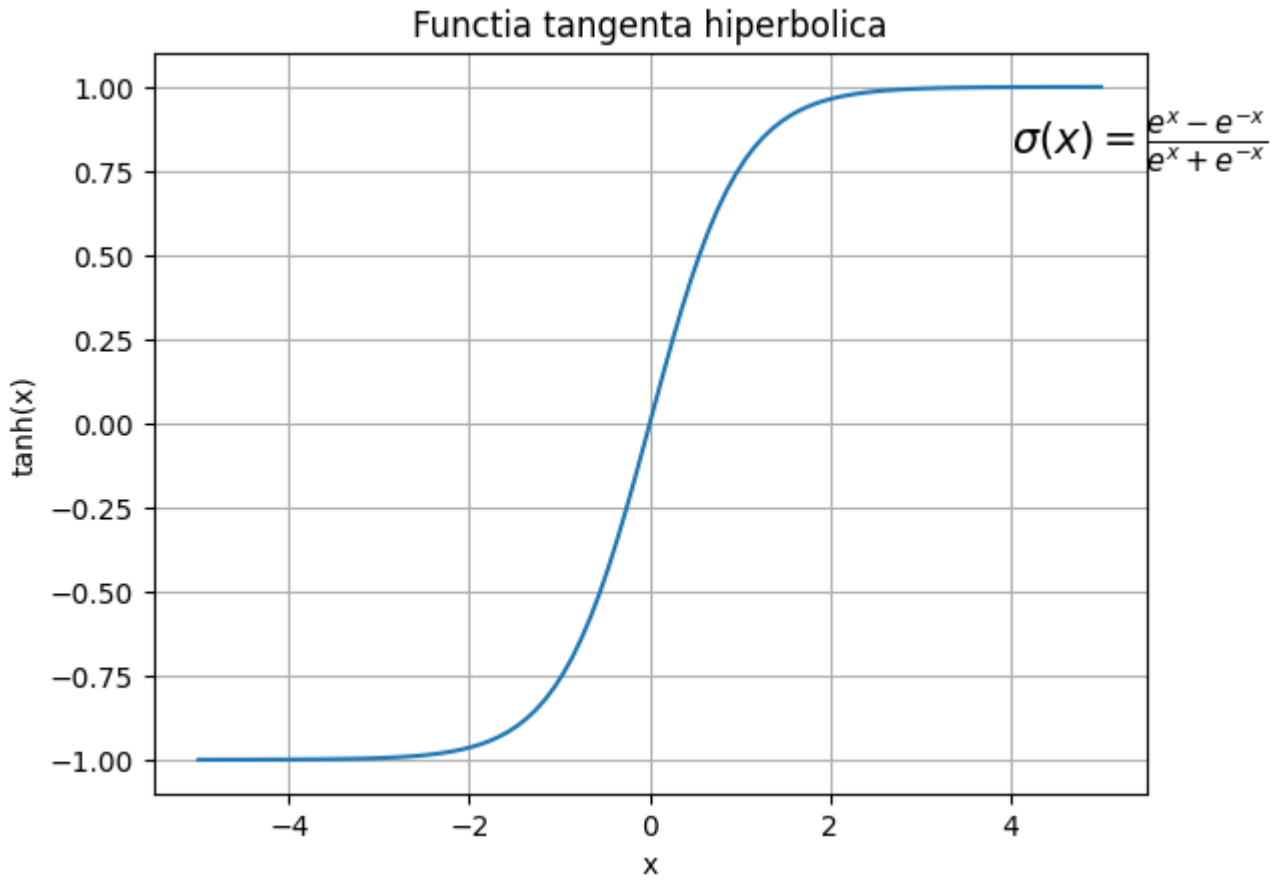
Are proprietatea ca este definita pe R cu valori in (-1,1).
tanh(0) = 0 care este punct de inflexiune pt graficul functiei.

Functia tangenta hiperbolica (tanh)

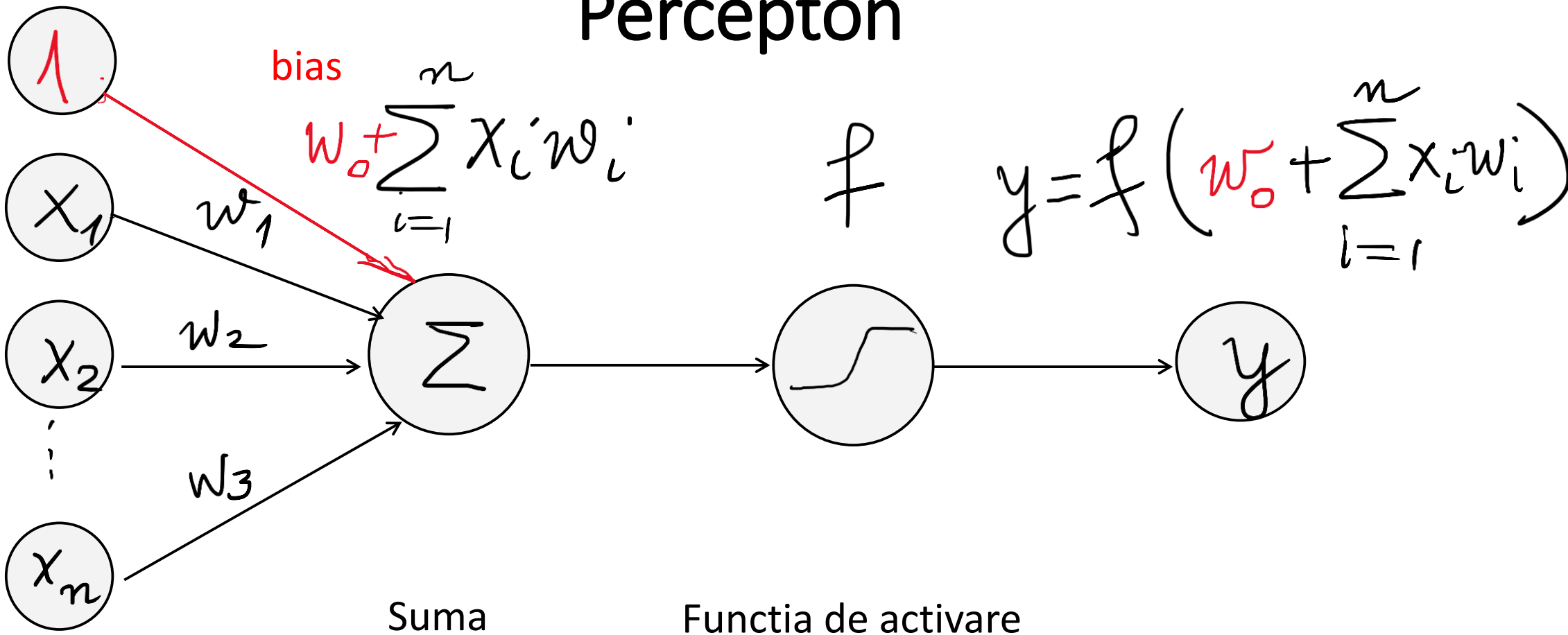
$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

tanh: R -> (-1,1)

tanh(0) = 0



Percepton


1
bias
$w_0 + \sum_{i=1}^{n} x_i w_i$
f
$y = f\left(w_0 + \sum_{i=1}^{n} x_i w_i\right)$
x_1
w_1
x_2
w_2
w_3
x_n
Σ
y
Suma
Functia de activare
input
ponderi
output

$$y = f\left(w_0 + \sum_{i=1}^{n} x_i w_i\right)$$

$(1 \times m) \cdot (n \times 1)$

$(x_1, \dots, x_m) \begin{pmatrix} w_1 \\ \vdots \\ w_m \end{pmatrix}$

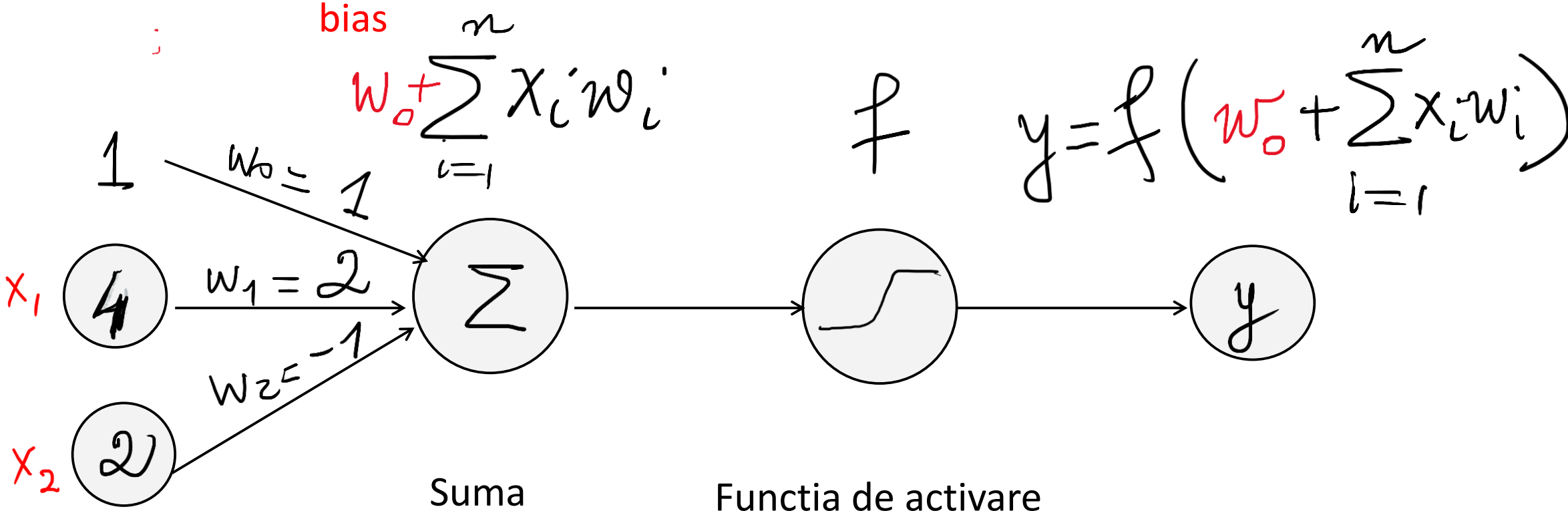
Notăm $X = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$, $w = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}$

$n \times 1$.

$$\sum_{i=1}^{n} x_i w_i = X^T \cdot w \quad \text{(înmulțire de matrici)}$$

$$y = f(w_0 + X^T \cdot w)$$

Exercitiu:



$X = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$, $W = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$, $b = 1$.

- X = [4, 2], W = [2, -1]

$X^T = (4 \quad 2)$ $W = \begin{pmatrix} 2 \\ -1 \end{pmatrix}$

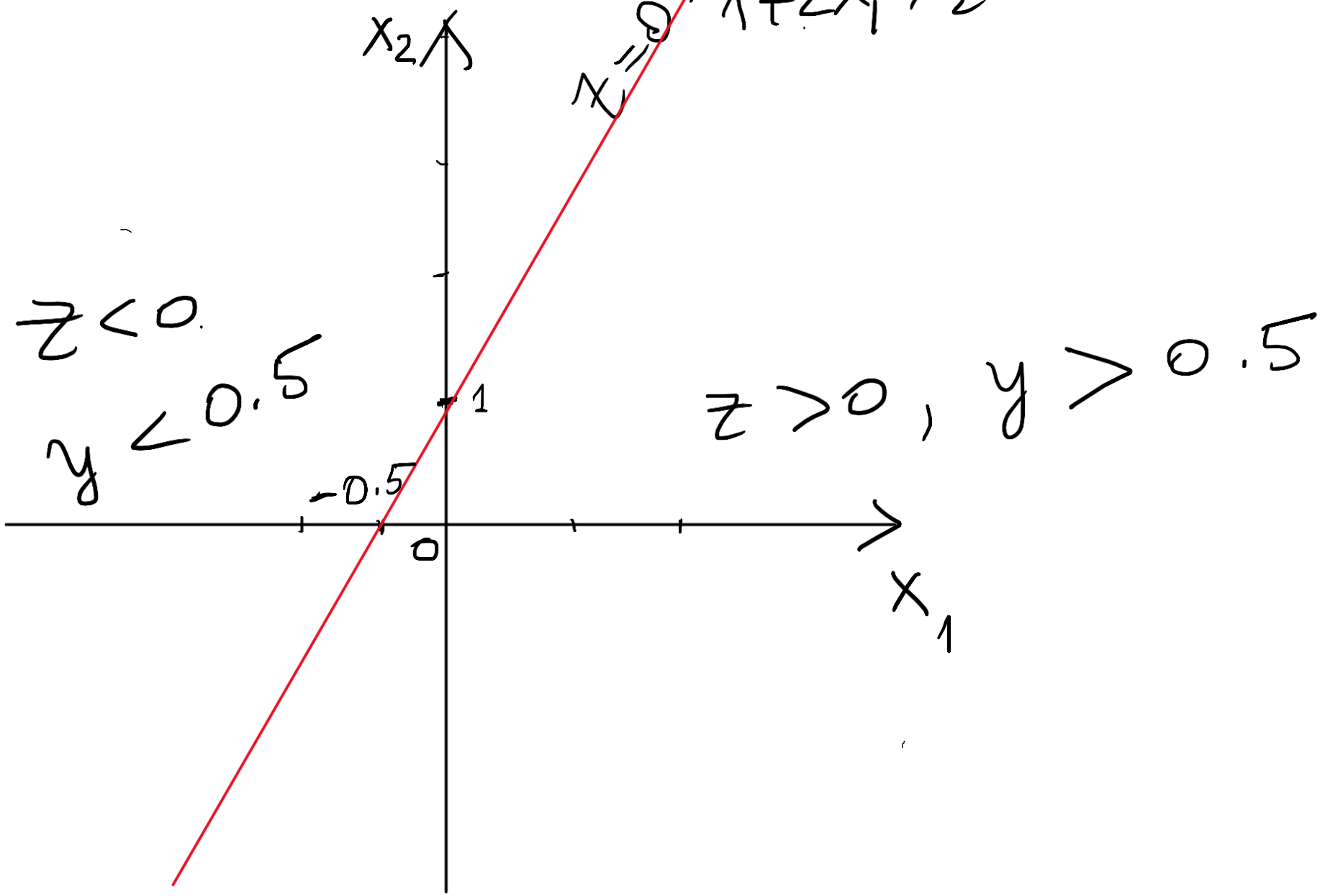
$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$X = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$, $W = \begin{bmatrix} 2 \\ -1 \end{bmatrix}$, $b = 1$. $\sigma(x) = \frac{1}{1+e^{-x}}$ output $= y$?

$$z = b + x^T \cdot W = 1 + \begin{bmatrix} 4 & 2 \end{bmatrix} \begin{bmatrix} 2 \\ -1 \end{bmatrix} = 1 + 4 \cdot 2 + 2 \cdot (-1) = 7.$$

$$y = \sigma(7) = 0.99908$$

$$z = 1 + 2x_1 - x_2$$



$z < 0$

$y < 0.5$

$z > 0, \; y > 0.5$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$y = \sigma(z) = \sigma(1 + x^T w)$$

Pt $x = \begin{bmatrix} 4 \\ 2 \end{bmatrix}$, $z = 1 + (4,2)\cdot\begin{pmatrix} 2 \\ -1 \end{pmatrix} = 1 + 8 - 2 = 7$.

$y = \sigma(z) = \sigma(7) =$ **0.99908894**

Pt $x = \begin{bmatrix} 2 \\ 4 \end{bmatrix}$ $z = 1 + (2,4)\begin{pmatrix} 2 \\ -1 \end{pmatrix} = 1 + 4 - 4 = 1$

$y = \sigma(z) = \sigma(1) =$ **0.7310586**

Dar pentru $X = \begin{pmatrix} -1 \\ 2 \end{pmatrix}$?

Dar pentru funcția ReLU ?

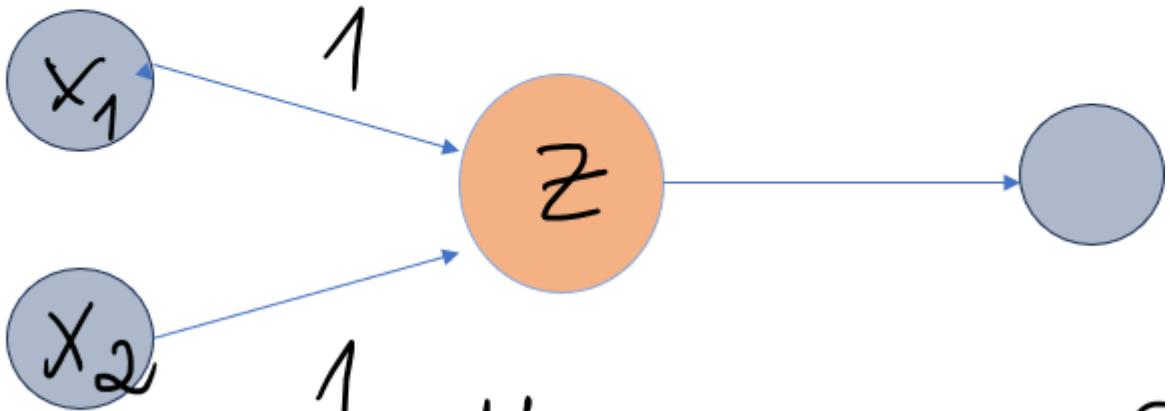
$$ReLU(x) = \begin{cases} 0, & x < 0 \\ x, & x \geq 0 \end{cases}.$$

$z = 7 \Rightarrow y = ReLU(z) = 7.$

$z = 1 \Rightarrow y = ReLU(1) = 1.$

- Exemplu – functia or: RxR -> {0,1}

  or(x,y) = x V y



$x_1 \vee x_2$.

$bias = -1$

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}.$$

x_1	x_2	z	y
0	0	-1	0
1	0	0	1
0	1	0	1
1	1	1	1

$z = -1 + x_1 + x_2$

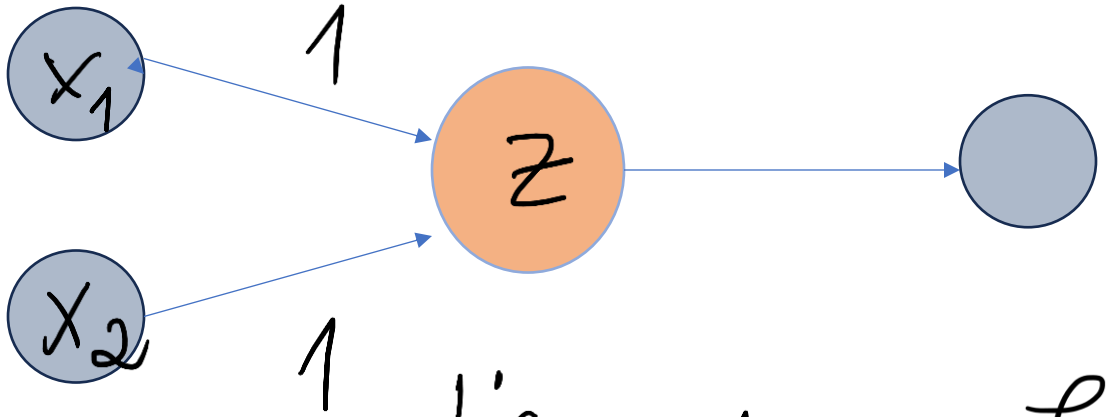
$y = f(z)$.

1	1	$z = -1 + 1 \cdot 1 + 1 \cdot 1 = 1$, $f(z) = 1$

x	y	x or y
0	0	$z = -1 + 1 \cdot 0 + 1 \cdot 0 = -1$, $f(z) = 0$
0	1	$z = -1 + 1 \cdot 0 + 1 \cdot 1 = 0$, $f(z) = 1$
1	0	$z = -1 + 1 \cdot 1 + 1 \cdot 0 = 0$, $f(z) = 1$

- Exemplu – functia or: RxR -> {0,1}

  or(x,y) = x V y

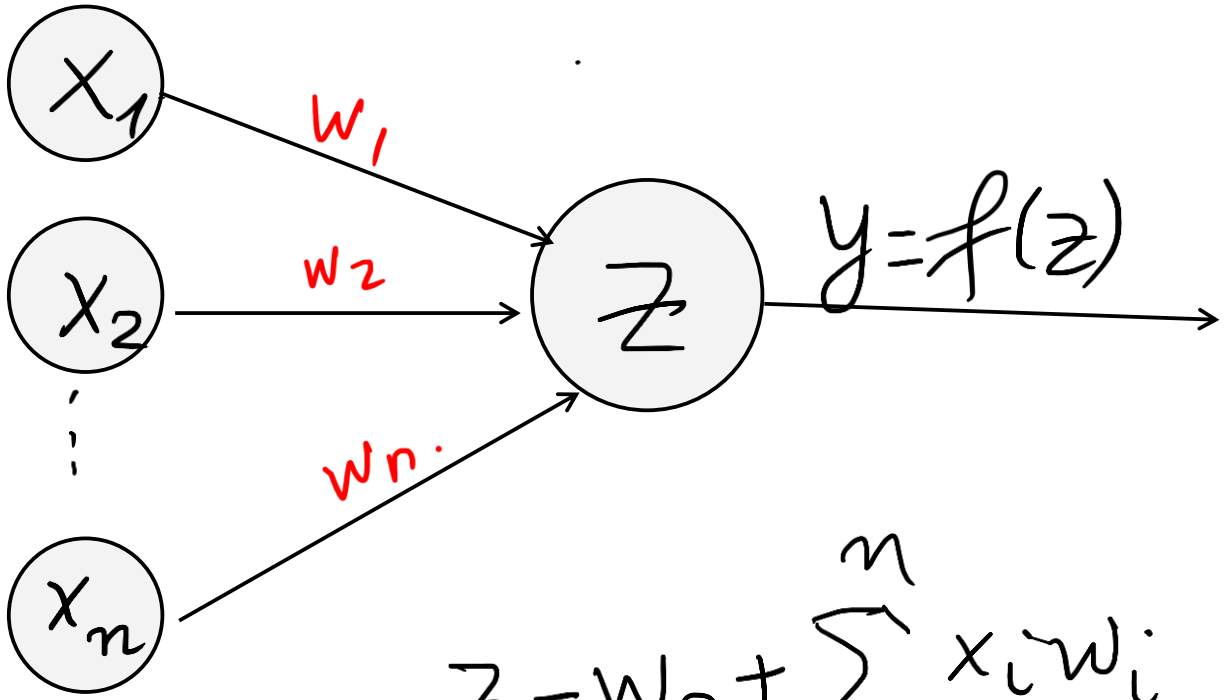


$y = f(z)$

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

treapta

Diagrama percepton

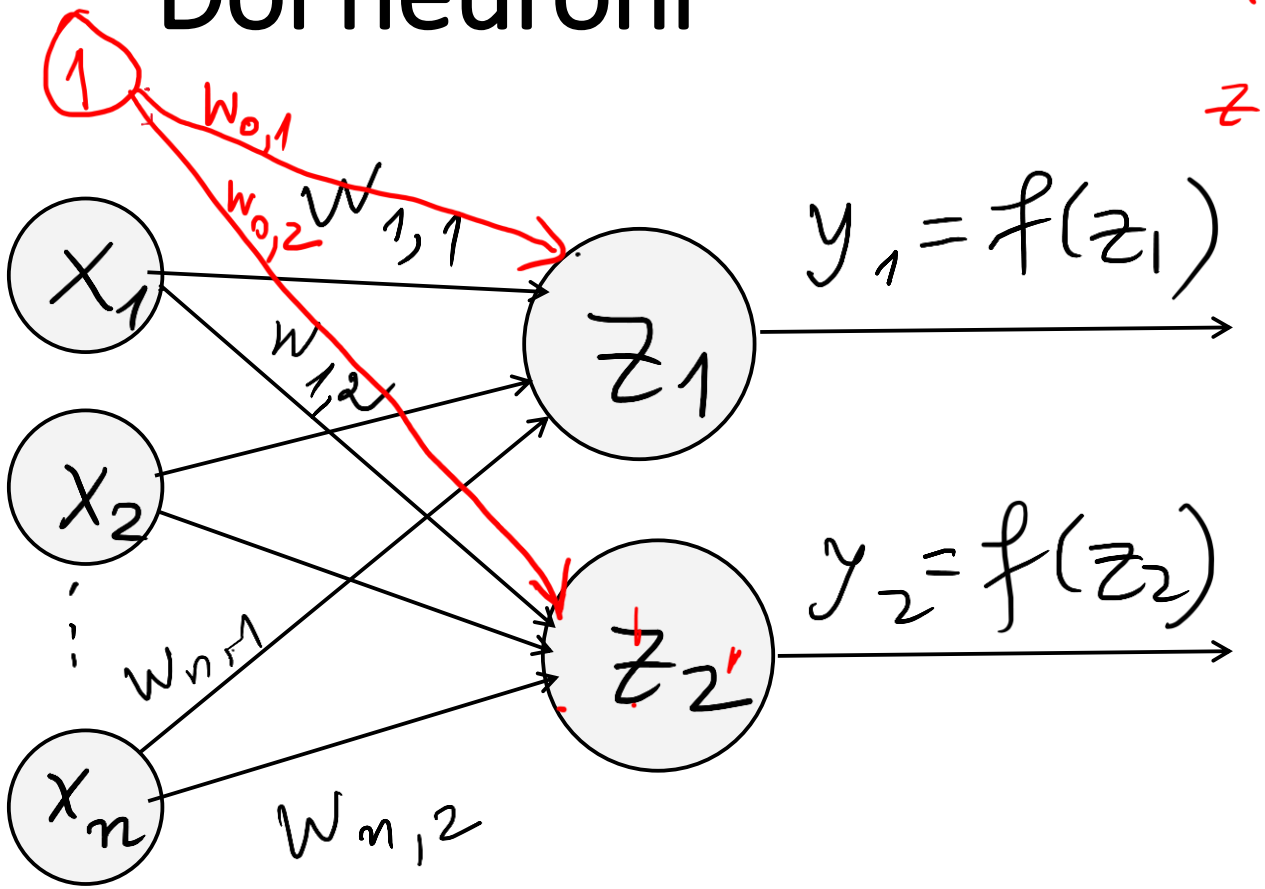


$$z = w_0 + \sum_{i=1}^{n} x_i w_i$$

$$z = w_0 + x^T w$$

Pt ca perceptronul este format dintr-un neuron si intr-o retea avem mai multi neuroni interconectati, simplificam diagrama astfel. Stim ca avem la bias, ponderi, si ca outputul se obtine prin aplicarea unei functii de activare.

Doi neuroni



$z_1 = w_{0,1} + x_1 w_{1,1} + \dots + x_n \cdot w_{n,1}$

$z_1 = w_{0,1} + x^T \cdot \begin{bmatrix} w_{1,1} \\ \vdots \\ w_{n,1} \end{bmatrix}$

$z_2 = w_{0,2} + x^T \cdot \begin{bmatrix} w_{1,2} \\ \vdots \\ w_{n,2} \end{bmatrix}$

pt x_i: $w_{i,1}$ → z_1, $w_{i,2}$ → z_2

$$z_j = w_{0,j} + \sum_{i=1}^{n} x_i w_{i,j} \quad \text{pt } j = 1,2$$

$$X = \begin{pmatrix} x_1 \\ \vdots \\ x_n \end{pmatrix} \qquad W = \begin{bmatrix} w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \\ \vdots & \vdots \\ w_{n,1} & w_{n,2} \end{bmatrix}$$

$n \times 2$ matrice de ponderi

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} w_{0,1} + \sum_{i=1}^{n} x_i \cdot w_{i,1} \\ w_{0,2} + \sum_{i=1}^{n} x_i w_{i,2} \end{bmatrix} =$$

$$z = \underline{W_0} + X^T W$$

bias (de obicei notat b).

$$W_0 = \begin{bmatrix} w_{0,1} \\ w_{0,2} \end{bmatrix}$$

b = vector $n \times 1$

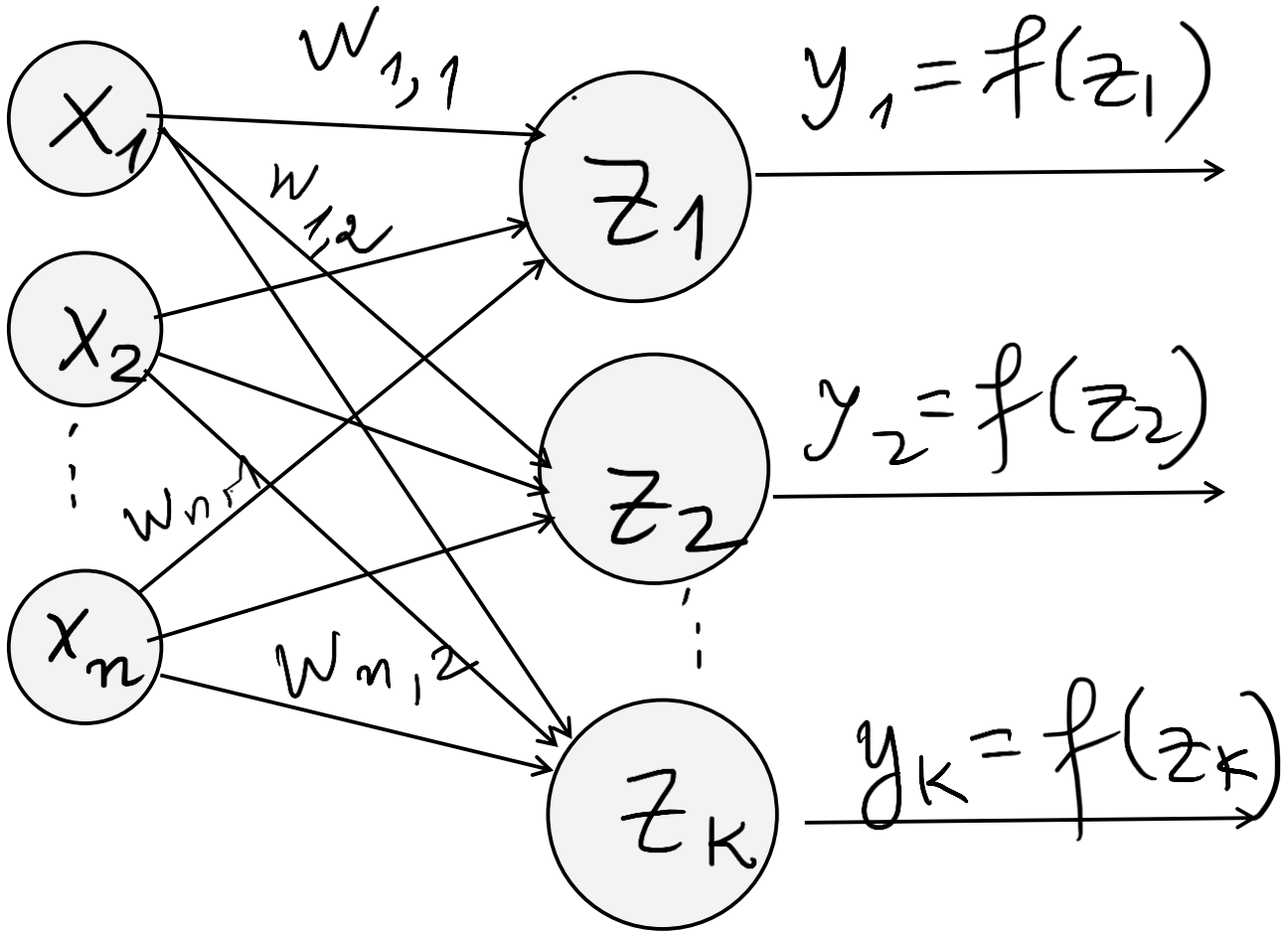
$$z = b + x^T \cdot w$$

(W = matricea ponderilor, b = vector bias, X = vector input)

$$Y = f(z) = \begin{bmatrix} f(z_1) \\ f(z_2) \end{bmatrix}$$

output

Mai multi neuroni



Retea densa pt ca fiecare valoare de intrare este conectata cu un nod de pe strat (layer).

W de dim

$n \times K$

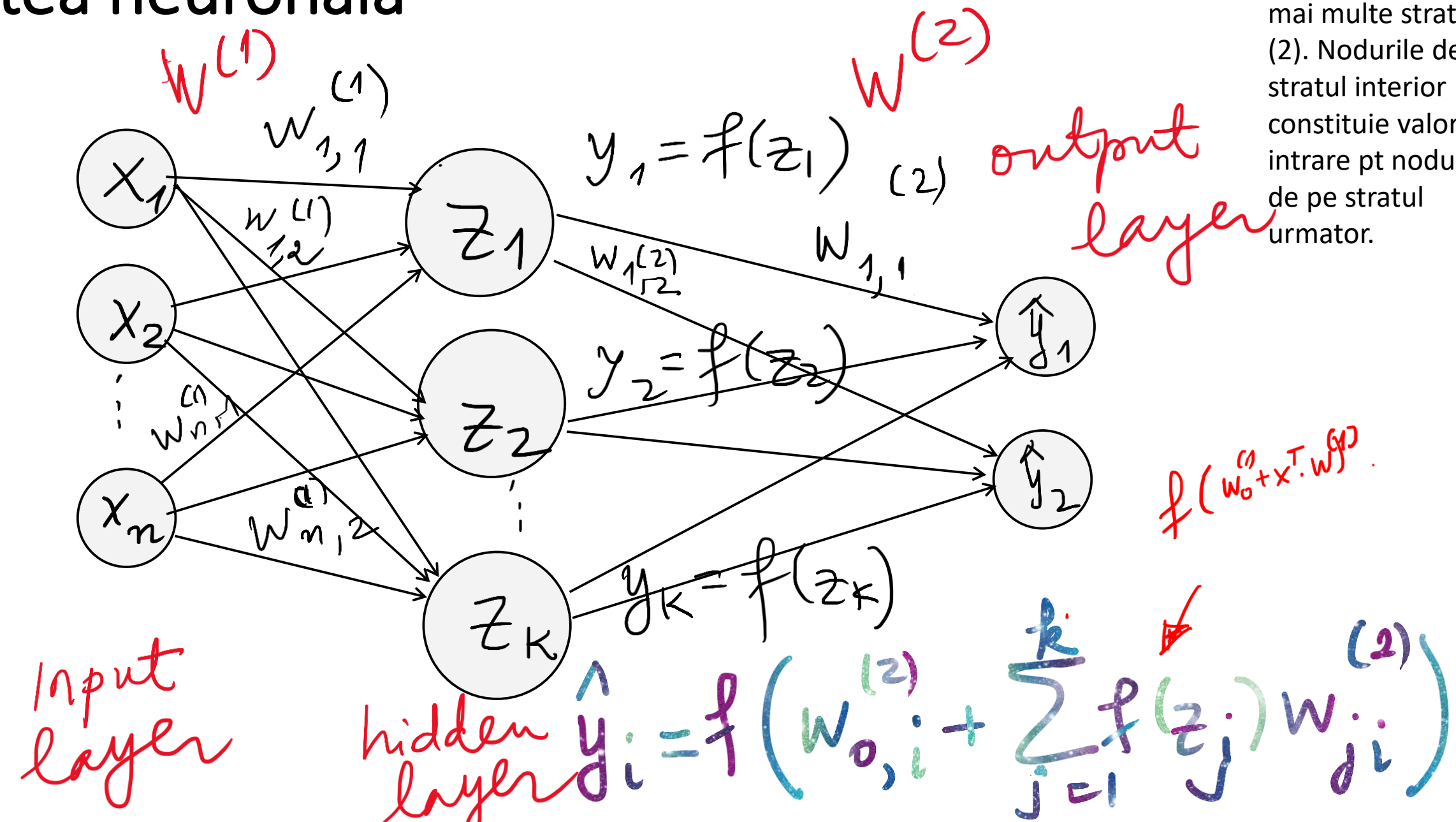
Pasii:

Calculeaza: $z = b + x^T \cdot w$

Aplica functia de activare : $f(z)$

Obtine output : $y = f(z)$

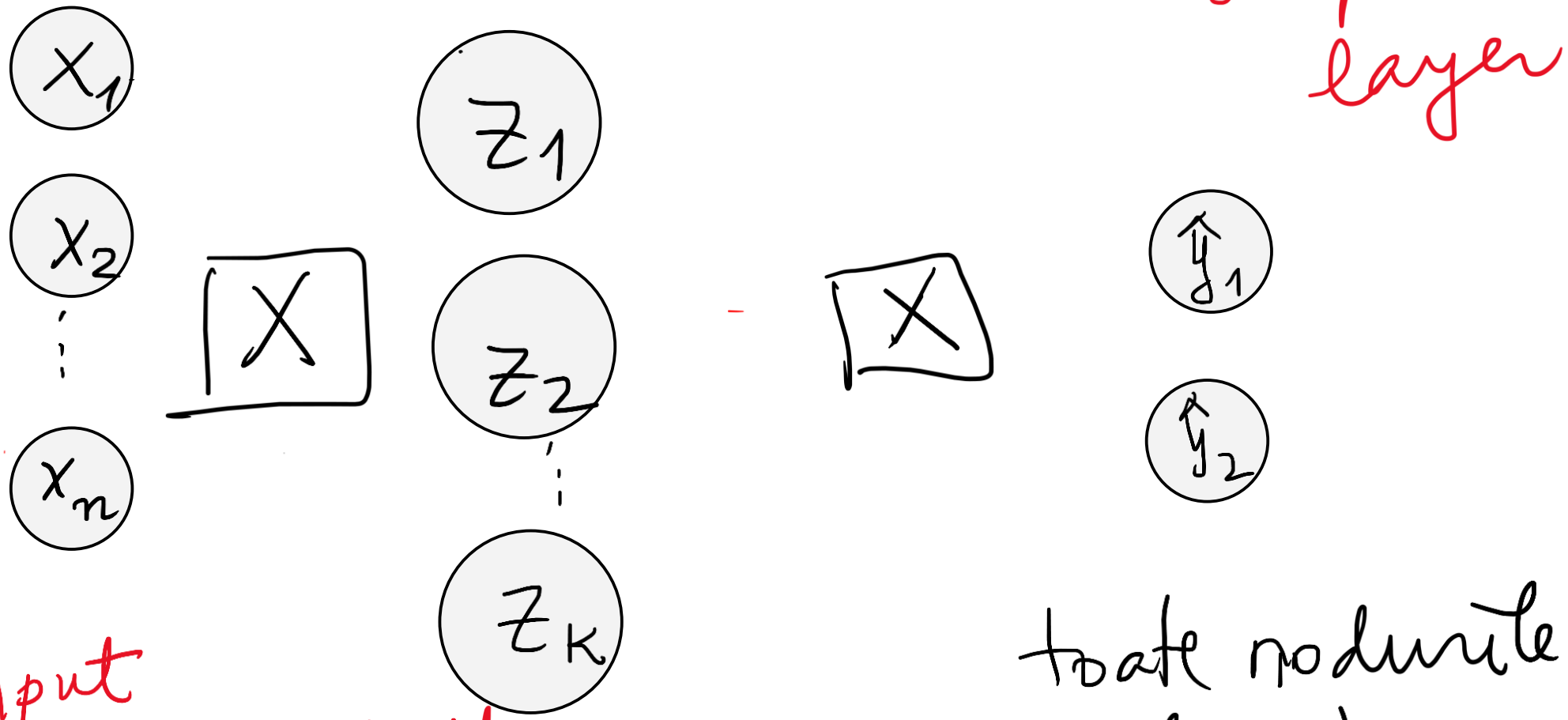
Retea neuronala



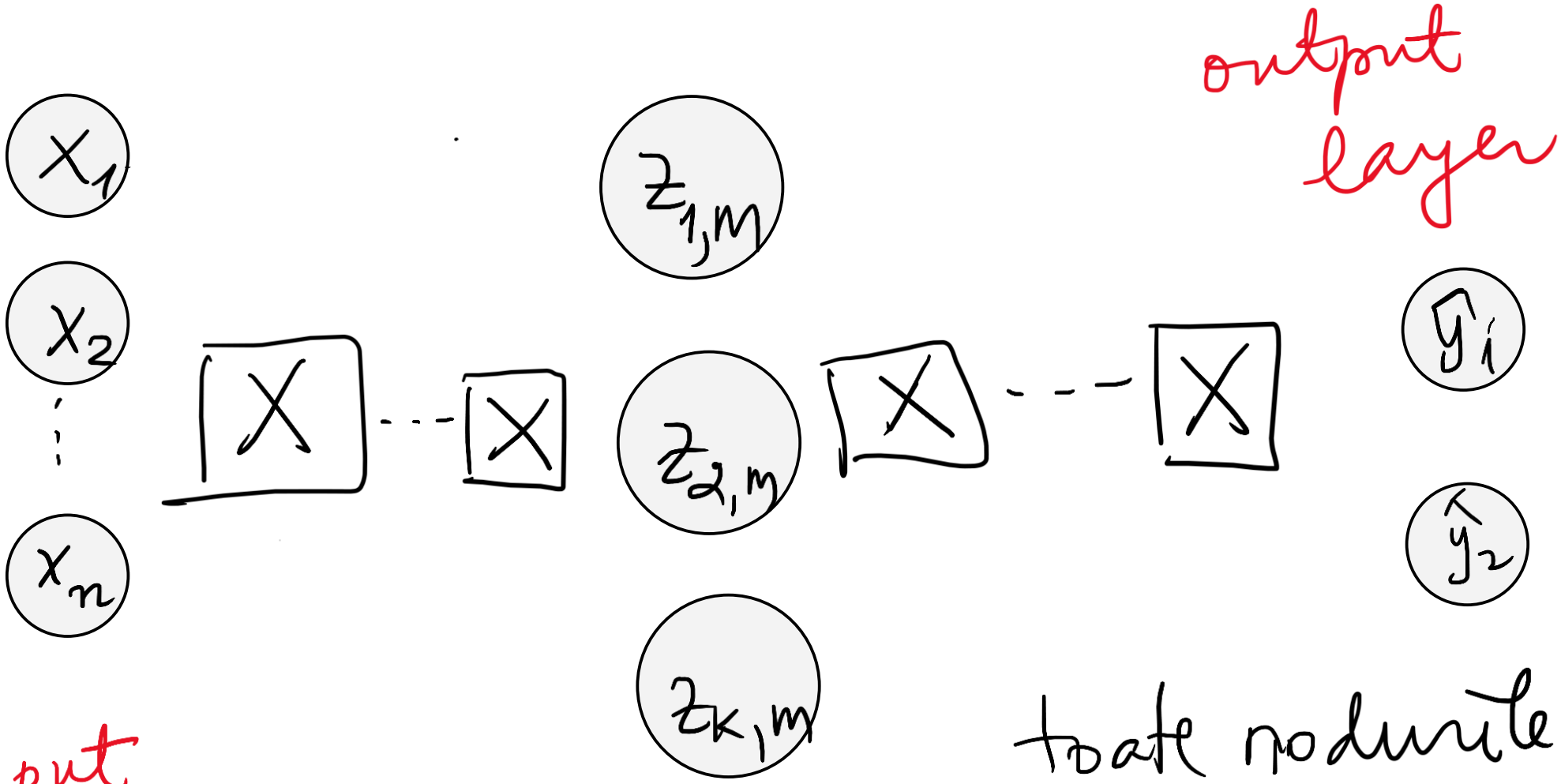
Vrem sa descriem acum o retea cu mai multe straturi. (2). Nodurile de pe stratul interior constituie valori de intrare pt nodurile de pe stratul urmator.

Multioutput perceptron

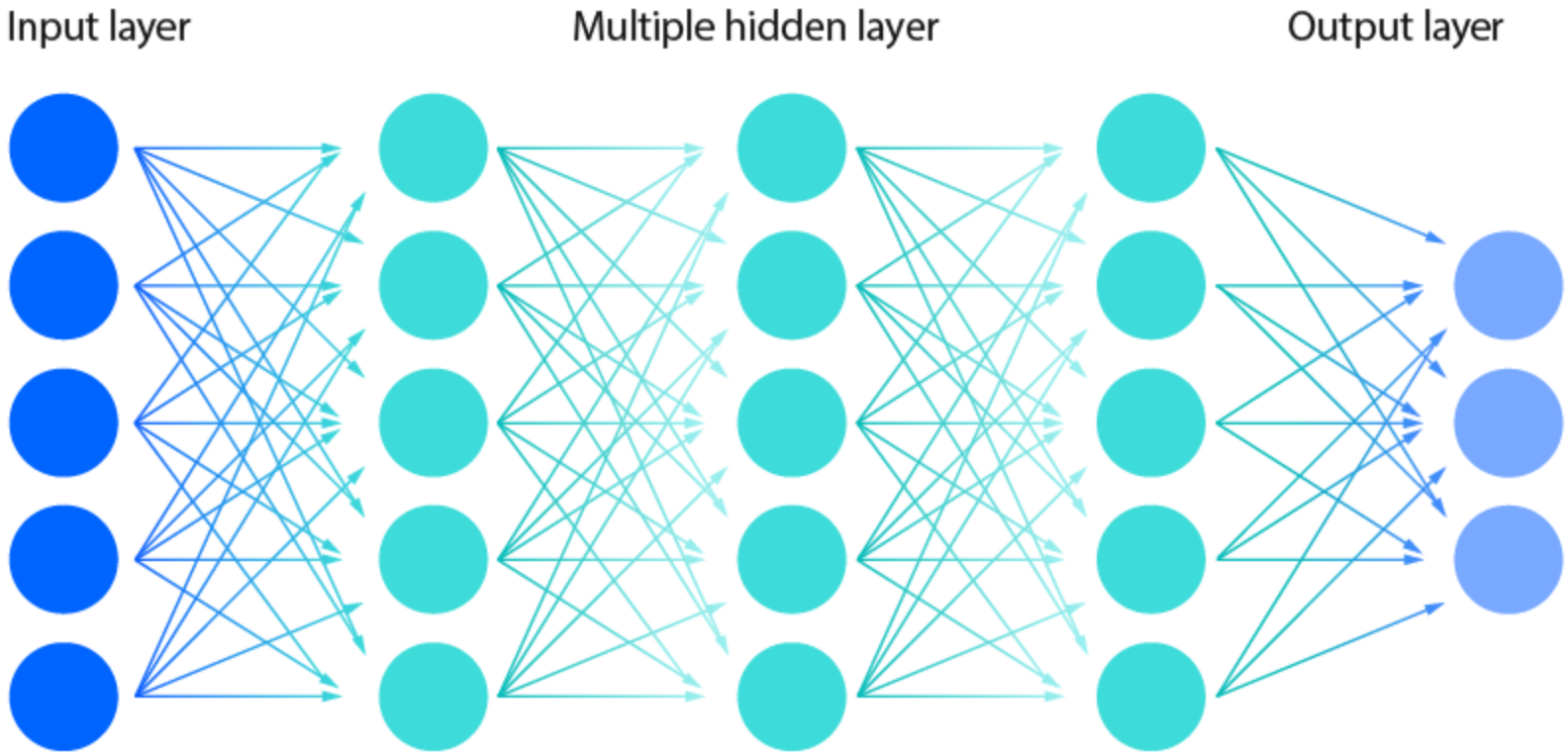
Reprezentam schematic facptul ca toate nodurile sunt interconectate.



Retea neuronala de tip deep



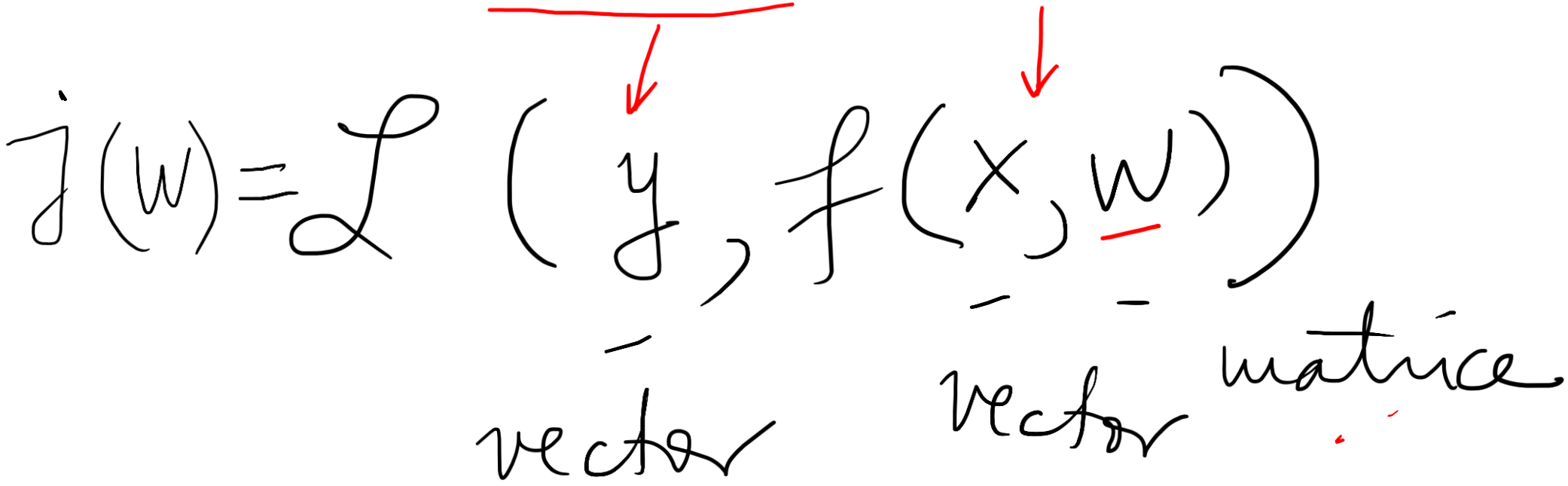
Deep neural network



Sursa: https://www.ibm.com/topics/neural-networks

Functia cost

- Se introduce functia cost (loss) sau pierdere, care va identifica ce diferenta este intre valoarea reala si valoarea de iesire.



Reteaua asa cum am descries-o pana acum a fost de tipul forward adica se pleaca de la niste date de intrare, se adauga mai multe straturi cu un nr de neuroni interconectati, pana cand se ajunge la un output. Acesta poate sa fie aproape sau nu de valoarea reala. De ce mai multe ori, aceste predictii initiale sunt departe de cele valorile reale. Unul din motive fiind si acela ca nu stim ce ponderi sa alegem pentru obtinerea valorilor de iesire. Cum masuram aceasta? Cum se face predictia?

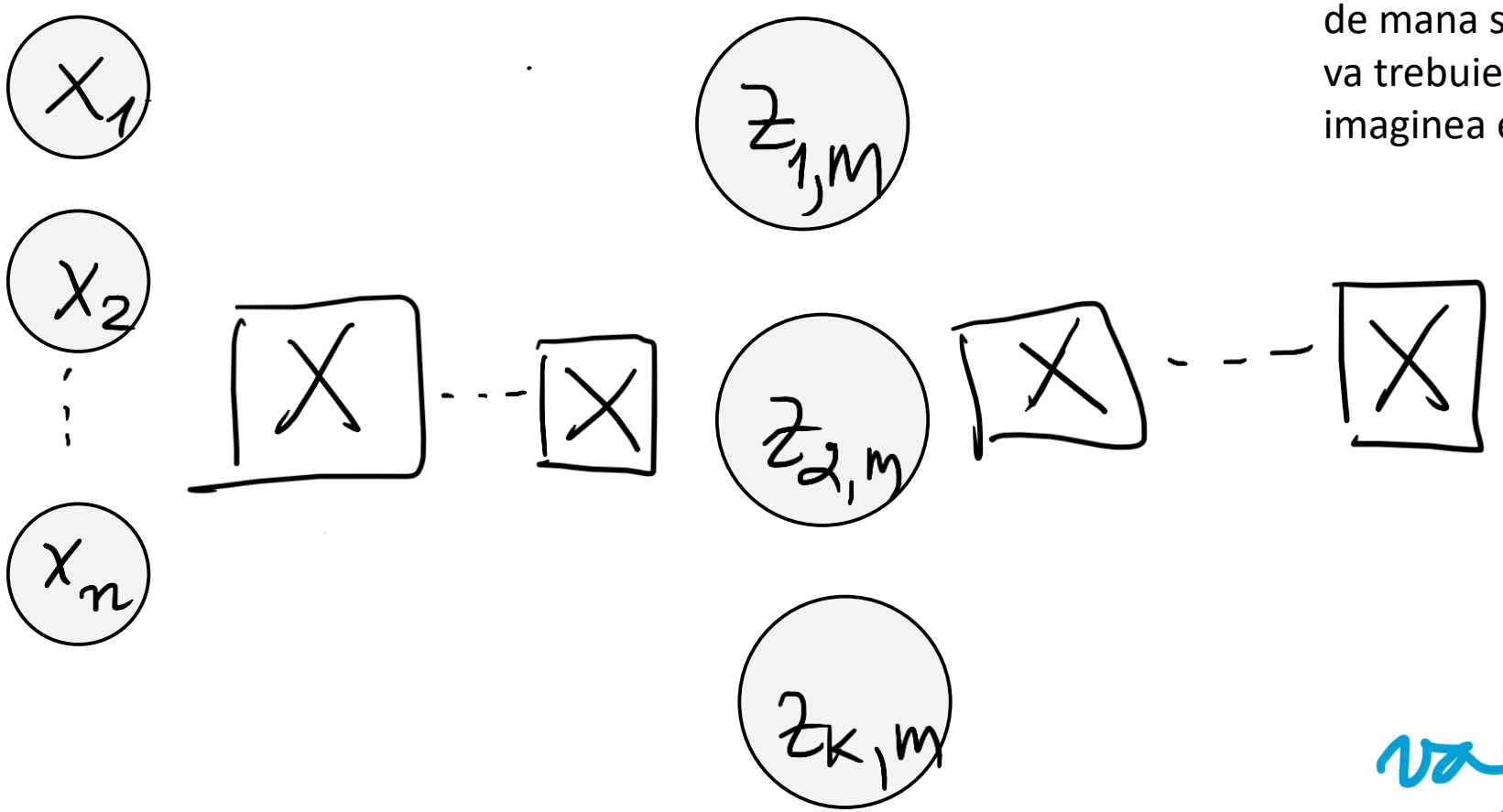
Exemplu

- Se dau la intrare niste imagini care vor fi identificate ca x sau 0.

Exemplu:

Sa se estimeze pretul unui apartament daca se cunoaste suprafata si nr de camere plecand de la o multime de training.

Se dau imagini de x si 0 scrise de mana si retea neuronala va trebuie sa identifice daca imaginea este un x sau un 0.



input

(80,3)

valoarea estimată 12345

val reală 100000

Functia cost

Se face media tuturor functiilor cost calculate pentru fiecare valoare de intrare comparand ceea ce a fost obtinut adica valoarea estimate cu ceea ce ar fi trebuit sa obtinem.

$$J(w) = \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\left(y^{(i)}, f(x^{(i)}, w)\right)$$

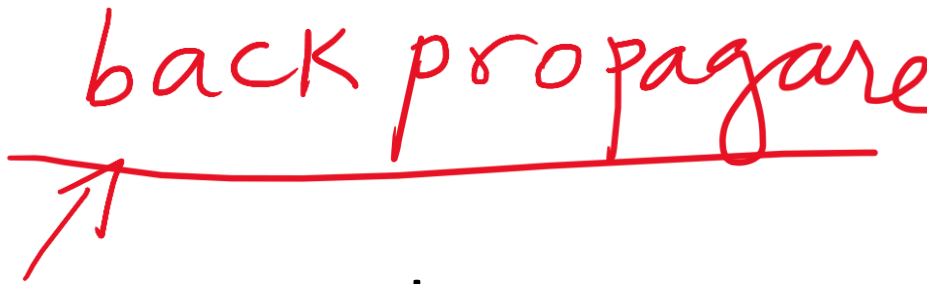
$y^{(i)}$: vector; $x^{(i)}$: vector; w: matrice

Metoda gradientului

Sa gasim ponderile W astfel incat J(W) sa fie minim.

Metoda gradientului este una din metodele folosite pentru a minimiza functia cost.

(vezi metoda de anul trecut de la cursul de DM si ML)



Dupa ce gasim aceste ponderi, se repeta calculele in retea cu noile ponderi.

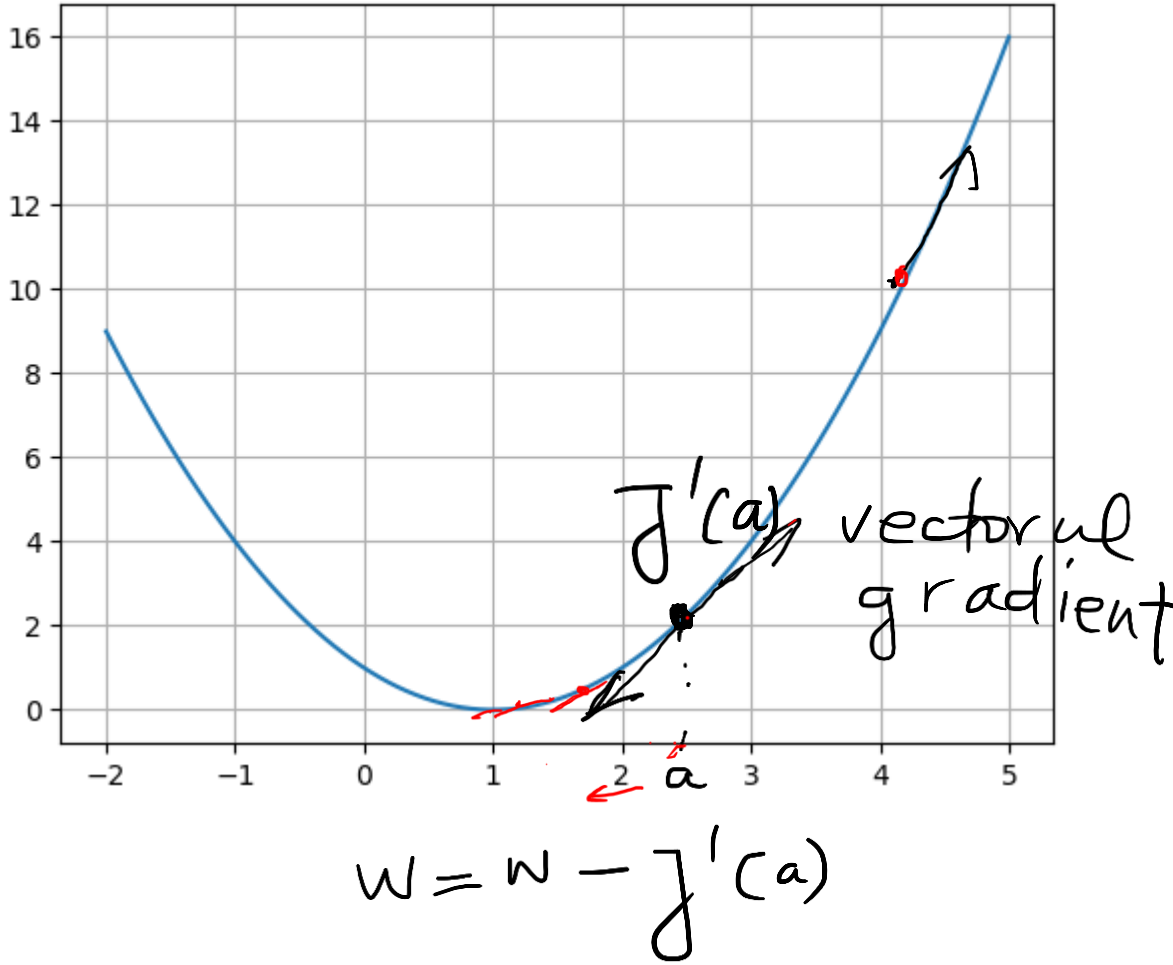
Ft o functie $F : \mathbb{R}^p \to \mathbb{R}$.

numim gradient

$$\nabla F = \begin{bmatrix} \frac{\partial F}{\partial x_1} \\ \vdots \\ \frac{\partial F}{\partial x_p} \end{bmatrix}$$

Gradient

- Pentru o functie definita pe R
- Gradient = Panta dreptei tangente la un punct de pe graficul functiei
- J’(a) vectorul gradient in directia de crestere a functiei
- Luam
- –J’(a)
- Pentru minimizare trebuie sa ne deplasam in sens opus vectorului gradient



Gradient

$$\nabla J(w) = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \frac{\partial J}{\partial w_2} \end{bmatrix} = \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix}$$

de ex

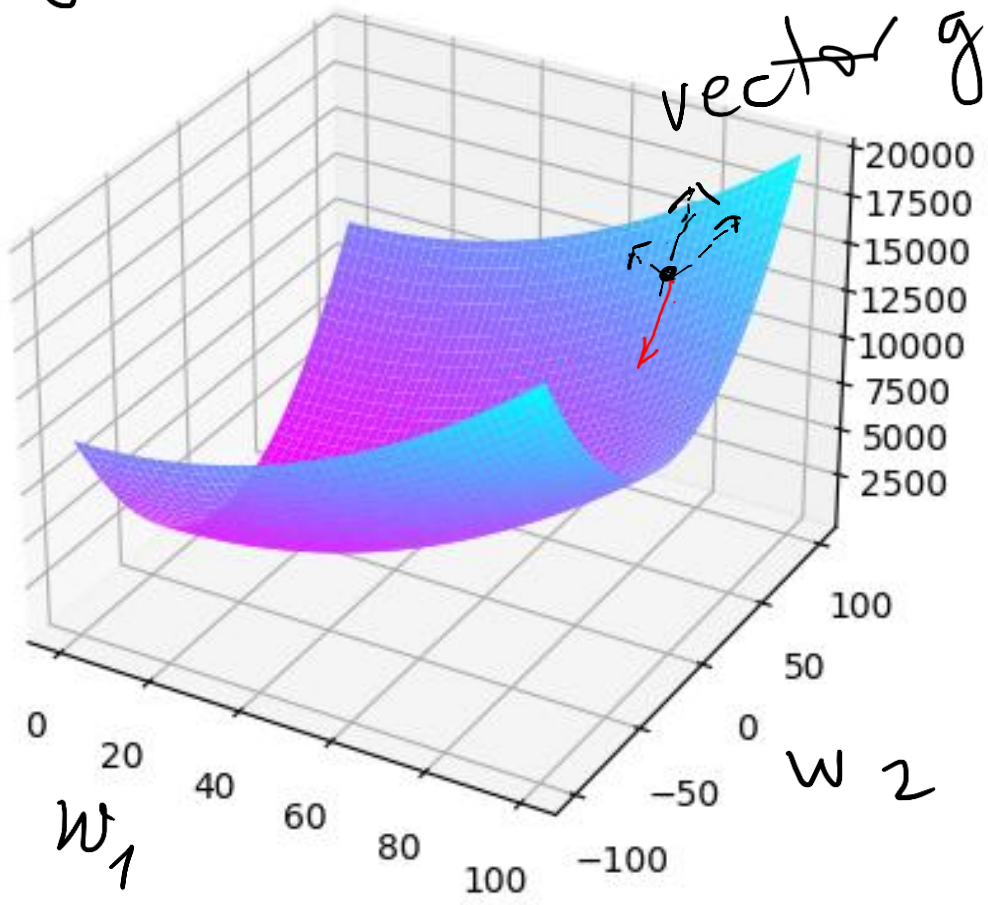
$$J(w_1, w_2) = w_1^2 + w_2^2$$

vector gradient

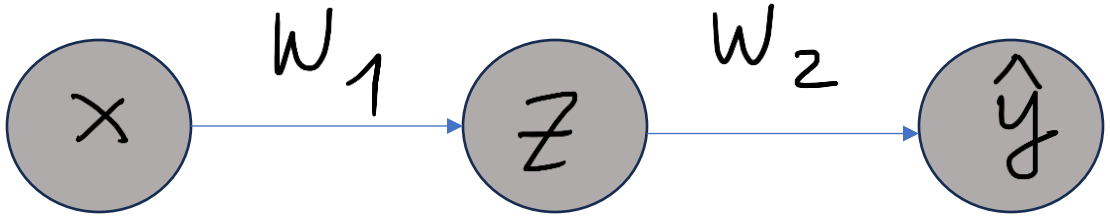
$$W = W - \begin{bmatrix} 2w_1 \\ 2w_2 \end{bmatrix}$$

- Pentru o functie definita pe R^2

- Gradient = Planul tangent la un punct de pe curba ce reprezinta functia



$$W = W - \nabla J(w_1, w_2)$$



$$\hat{y} = f(w_2 \cdot z)$$

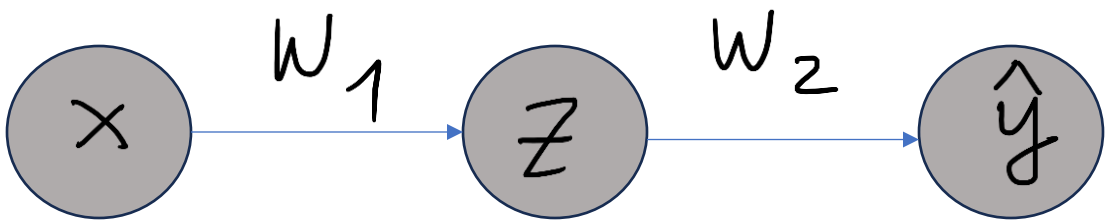
$$z = f(w_0 + w_1 x)$$

functia cost

$$J(w) = \mathcal{L}(y, \hat{y})$$

f = fct. de activare

gradientul: $\nabla J(w) = \frac{\partial J(w)}{\partial w} = \begin{bmatrix} \frac{\partial J}{\partial w_1} \\ \frac{\partial J}{\partial w_2} \end{bmatrix}$

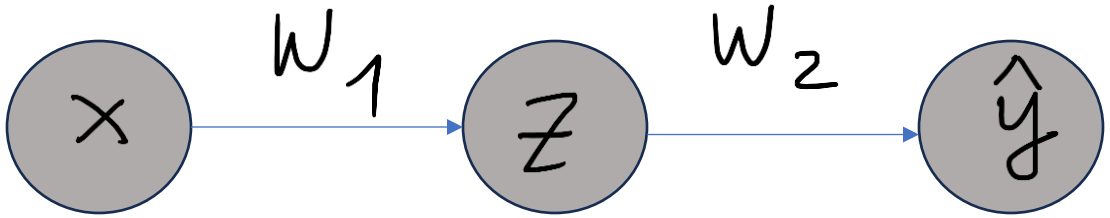


$$\hat{y} = f(w_2 \cdot z)$$

$$z = f(w_0 + w_1 x)$$

$$\frac{\partial J}{\partial w_2} = \frac{\partial J(w)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2}$$

$$\frac{\partial \hat{y}}{\partial w_2} = f'(w_2 \cdot z) \cdot z$$



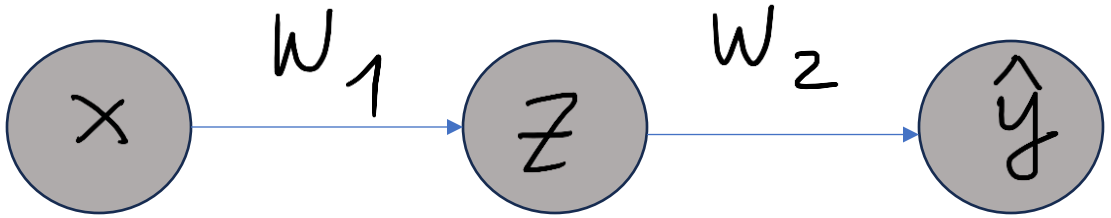
$$\hat{y} = f(w_2 \cdot z)$$

$$z = f(w_0 + w_1 x)$$

$$\frac{\partial J}{\partial w_1} = \frac{\partial J(w)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

$$\frac{\partial \hat{y}}{\partial z} = f'(w_2 \cdot z) \cdot w_2 \qquad \frac{\partial z}{\partial w_1} = f'(w_0 + w_1 x) \cdot x$$

f = fct. de activare



$$\hat{y} = f(w_2 \cdot z)$$

$$z = f(w_0 + w_1 x)$$

$$J(w) = \mathcal{L}(y, \hat{y})$$

$$\frac{\partial J}{\partial w_2} = \frac{\partial J(w)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2}$$

$$\frac{\partial J}{\partial w_1} = \frac{\partial J(w)}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w_1}$$

Algoritmul coborarii de gradient (Metoda gradientului)

Se alege $W^{(0)}$ valoare inițială (de obicei normal distribuită)

$k = 0$

Repetă

calculează $\frac{\partial J(W)}{\partial W}$

$$W^{(k+1)} = W^{(k)} - \eta_k \frac{\partial J(W)}{\partial W}$$

until $k > MAX$ or $d(W^{(k)}, W^{(k+1)}) < \varepsilon$

MAX = nr maxim iterații

Algoritmul coborarii de gradient (Metoda gradientului)

Se alege $w^{(0)}$ valoare inițială (de obicei normal distribuită)

$k = 0$

repetă

calculează $\frac{\partial J(w)}{\partial w}$

$w^{(k+1)} = w^{(k)} - \eta_k \frac{\partial J(w)}{\partial w}$

rata de învățare

until $k > MAX$ or $d(w^{(k+1)}, w^{(k)}) < \varepsilon$

MAX = nr maxim iterații

Observatii:

- η_k un parametru f important (learning rate) rata de invatare
- Rata de învățare mai mică, sunt necesare mai multe iteratii sau poate atinge iterația maximă înainte de a ajunge la punctul optim
- Dacă rata de învățare este prea mare, algoritmul poate să nu conveargă la punctul optim (salt în jur) sau chiar să se abată complet.
-

- https://miro.medium.com/v2/resize:fit:1400/1*v5bc1TzeMKpzTAgorjOoHQ.gif
- https://miro.medium.com/v2/resize:fit:1400/format:webp/1*GR914FuA4pVTTXEpVDJ2Ng.png
- https://miro.medium.com/v2/resize:fit:640/1*Z3wjHCFAehYXYG9lOiqiEw.gif
- https://towardsdatascience.com/gradient-descent-algorithm-a-deep-dive-cf04e8115f21

Algoritmul de propagare înapoi a erorii (backpropagation)

- Alege ponderi in mod aleatoriu (pot fi alese ca sa aiba distributie normala)
- Repeat
- Calculeaza gradientul erorii in fiecare punct – functia cost
- Actualizeaza ponderile conform metodei gradientului\
- Until se obtine convergenta

Clasificare binara –functia cost entropie

$$J(w) = -\frac{1}{m}\sum_{i=1}^{m} y^{(i)} \log\left(f(x^{(i)}, w)\right) + (1 - y^{(i)}) \log\left(1 - f(x^{(i)}, w)\right)$$

Loss = tf.reduce_mean(tf.nn.softmax_cross_entropy_with_logits(y,predicted)

Functia cost - squared mean error

$$J(w) = \frac{1}{n} \sum_{i=1}^{n} \left(y^{(i)} - f(x^{(i)}, w) \right)^2$$

Loss = tf.reduce_mean(tf.square(tf.substract (y, predicted)))
Loss = tf.keras.losses.MSE(y, predicted)